From Jacobi/SOR to a Direct FFT Solver for the 2D Laplace Equation

OpenMP, OpenACC, cuFFT, DST-I, and performance lessons

HPC Lessons

April 27, 2026

Problem statement

We solve Laplace's equation on a square:

$$u_{xx} + u_{yy} = 0, \qquad 0 < x < 1, \quad 0 < y < 1.$$

Boundary conditions:

$$u(x,0) = \sin(\pi x), \qquad u(x,1) = e^{-\pi}\sin(\pi x),$$

$$u(0,y) = 0, \qquad u(1,y) = 0.$$

Analytical validator:

$$u_{\text{exact}}(x,y) = \sin(\pi x)e^{-\pi y}.$$

Important: the analytical solution is used only for validation, not by the direct solver.

Uniform-grid finite-difference problem

Interior grid points:

$$i = 1, \ldots, N_x, \qquad j = 1, \ldots, N_y.$$

Five-point discrete Laplacian:

$$(Lu)_{i,j} = \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{\Delta x^2} + \frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{\Delta y^2}.$$

For Laplace:

$$(Lu)_{i,j} = 0.$$

Known boundary values are moved to the right-hand side for the interior unknowns.

Jacobi: correct but slow

Jacobi uses two arrays:

$$u_{i,j}^{(m+1)} = \frac{1}{4}\left(u_{i+1,j}^{(m)} + u_{i-1,j}^{(m)} + u_{i,j+1}^{(m)} + u_{i,j-1}^{(m)}\right).$$

OpenMP safety:

- read only from `u`
- write only to distinct entries of `u_new`
- reduction for error

So Jacobi is thread-safe, but convergence is slow.

Jacobi pointer swap

Avoid copying arrays after each iteration:

```
temp  => u
u     => u_new
u_new => temp
```

This is an $O(1)$ pointer swap, instead of an $O(N^2)$ copy.
For GPU/OpenACC, pointer swapping is possible but easier to manage with explicit buffers:

```
real(8), allocatable, target :: u_buf(:,:), unew_buf(:,:)
real(8), pointer, contiguous :: u(:,:), u_new(:,:), tmp(:,:)

u     => u_buf
u_new => unew_buf
```

Red-black SOR

SOR updates in place:

$$u_{i,j}^{new} = u_{i,j}^{old} + \omega \left[\frac{1}{4} \left(u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} \right) - u_{i,j}^{old} \right].$$

Red-black ordering avoids dependency conflicts:

$$\text{red: } i + j \text{ even}, \qquad \text{black: } i + j \text{ odd}.$$

Each color can be updated in parallel because neighboring points belong to the opposite color.

SOR cost model

Let the grid be $N \times N$.
Cost per sweep:

$$O(N^2).$$

For elliptic problems, SOR needs approximately:

$$O(N) \quad \text{iterations}$$

with near-optimal relaxation.
Total cost:

$$O(N^3).$$

This is why a direct spectral method can dominate at large grid sizes.

Direct solver idea

The finite-difference operator is linear:

$$Lu = f.$$

If we transform into an eigenbasis of L:

$$L\phi_{k,\ell} = \lambda_{k,\ell}\phi_{k,\ell},$$

then the system decouples:

$$\widehat{u}_{k,\ell} = \frac{\widehat{f}_{k,\ell}}{\lambda_{k,\ell}}.$$

Algorithm:

$$f \xrightarrow{\text{transform}} \widehat{f} \xrightarrow{\text{divide}} \widehat{u} \xrightarrow{\text{inverse transform}} u.$$

Dirichlet boundaries require sine modes

For homogeneous Dirichlet boundaries, the eigenvectors are sine functions:

$$\phi_{k,\ell}(i,j) = \sin\left(\frac{k\pi i}{N_x+1}\right)\sin\left(\frac{\ell\pi j}{N_y+1}\right).$$

The correct transform is the DST-I, not a periodic FFT.

Periodic FFT diagonalizes the periodic Laplacian. Dirichlet BC define a different operator.

Discrete Laplacian eigenvalues

For the 1D second-difference operator with Dirichlet boundaries:

$$\lambda_k^x = \frac{2\left(\cos\frac{k\pi}{N_x+1} - 1\right)}{\Delta x^2}.$$

Similarly in y:

$$\lambda_\ell^y = \frac{2\left(\cos\frac{\ell\pi}{N_y+1} - 1\right)}{\Delta y^2}.$$

For the 2D operator:

$$\lambda_{k,\ell} = \lambda_k^x + \lambda_\ell^y.$$

Then:

$$\widehat{u}_{k,\ell} = \frac{\widehat{f}_{k,\ell}}{\lambda_k^x + \lambda_\ell^y}.$$

How boundary conditions enter the RHS

Interior equation at the first interior row:

$$\frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{\Delta y^2} + \cdots = 0.$$

At $j = 1$, the neighbor $u_{i,0}$ is a known boundary value. Moving it to the RHS gives:

$$f_{i,1} = -\frac{u_{i,0}}{\Delta y^2}.$$

At the last interior row:

$$f_{i,N_y} = -\frac{u_{i,N_y+1}}{\Delta y^2}.$$

For this problem:

$$f_{i,1} = -\frac{\sin(\pi x_i)}{\Delta y^2}, \qquad f_{i,N_y} = -\frac{e^{-\pi}\sin(\pi x_i)}{\Delta y^2}.$$

Why complex values appear

The sine expansion is real:

$$u_{i,j} = \sum_{k,\ell} \widehat{u}_{k,\ell} \sin\left(\frac{k\pi i}{N_x+1}\right) \sin\left(\frac{\ell\pi j}{N_y+1}\right).$$

But cuFFT computes complex exponential transforms:

$$e^{i\theta} = \cos\theta + i\sin\theta.$$

DST-I is obtained by an odd extension. For an odd signal, the FFT is purely imaginary, so:

$$\mathrm{DST}(f)_k \propto \mathrm{Im}(\mathrm{FFT}(\tilde{f})_k).$$

Complex arrays are therefore temporary workspace only.

Odd extension for 1D DST-I

For data $f_1, \ldots, f_N$, create length $2(N+1)$ array:

$$\tilde{f}_0 = 0, \quad \tilde{f}_i = f_i, \quad \tilde{f}_{N+1} = 0, \quad \tilde{f}_{2(N+1)-i} = -f_i.$$

In code form:

```
work(1) = 0
work(i+1) = f(i)
work(next-i+1) = -f(i)
work(N+2) = 0
```

Then:

```
F = FFT(work)
fhat(i) = -0.5d0 * aimag(F(i+1))
```

Version 1: monolithic 2D DST through odd-odd extension

Your first FFT solver builds a full antisymmetric array:

$$\tilde{f} \in \mathbb{C}^{2(N_x+1)\times 2(N_y+1)}.$$

Then it applies one 2D FFT:

$$\widehat{f} = \mathrm{FFT}_{2D}(\tilde{f}).$$

This realizes:

$$\mathrm{DST}_x \mathrm{DST}_y(f).$$

Advantages:

- easiest to understand
- direct tensor-product construction

Disadvantage:

- large memory: roughly four times the physical grid in complex storage

Version 2: separable batched 1D DSTs

The 2D DST is separable:

$$\mathrm{DST}_{2D}(f) = \mathrm{DST}_x\left(\mathrm{DST}_y(f)\right).$$

So instead of one huge 2D extension, apply:

$$f \xrightarrow{\mathrm{DST}_x} F_x \xrightarrow{\mathrm{DST}_y} \widehat{f}.$$

Then solve:

$$\widehat{u}_{k,\ell} = \frac{\widehat{f}_{k,\ell}}{\lambda_k^x + \lambda_\ell^y}.$$

Then inverse in reverse order:

$$\widehat{u} \xrightarrow{\mathrm{DST}_y^{-1}} \xrightarrow{\mathrm{DST}_x^{-1}} u.$$

Linear algebra view

Let S_x and S_y be sine-transform matrices.
The discrete operator has Kronecker form:

$$L = L_x \otimes I + I \otimes L_y.$$

The sine matrices diagonalize the 1D operators:

$$S_x L_x S_x^{-1} = \Lambda_x, \qquad S_y L_y S_y^{-1} = \Lambda_y.$$

Therefore the 2D operator becomes diagonal:

$$\widehat{L}_{k,\ell} = \lambda_k^x + \lambda_\ell^y.$$

This is the mathematical reason the direct solver works.

Spectral division kernel

After the forward DSTs:

```
spec(i,j) = spec(i,j) / &
  ( 2.d0*(cos(pi*dble(i)/dble(nxi+1)) - 1.d0)/dx**2 + &
    2.d0*(cos(pi*dble(j)/dble(nyi+1)) - 1.d0)/dy**2 )
```

This is the whole solve:

$$\widehat{u}_{i,j} = \frac{\widehat{f}_{i,j}}{\lambda_i^x + \lambda_j^y}.$$

No iterations are needed.

Normalization in the batched DST implementation

Each 1D DST extraction uses:

$$\widehat{f}_k = -\frac{1}{2}\,\mathrm{Im}(\mathrm{FFT}(\tilde{f})_{k+1}).$$

You apply DST operations four times:

1. forward in x
2. forward in y
3. inverse in y
4. inverse in x

For your implementation, the final scaling is:

$$\mathrm{scale} = \frac{4}{(N_x+1)(N_y+1)}.$$

This is why the batched version needed the corrected scaling.

Memory comparison

For an $N \times N$ interior grid:

Implementation	Main temporary	Approx. size
Full 2D extension	$(2N) \times (2N)$ complex	$4N^2$ complex values
Batched 1D DST	$(2N) \times N$ complex	$2N^2$ complex values
Physical solution	$N \times N$ real	N^2 real values

The batched version cuts the main FFT temporary roughly in half and reduces workspace pressure.

Runtime comparison example

Example result for 4096×4096:

Method	Runtime	Error
Red-black SOR	367.4 s	1.69×10^{-4}
FFT direct solver	1.18 s	$1.74 \sim 10^{-8}$

Measured speedup:

$$\frac{367.4}{1.18} \approx 311.$$

The FFT solver is faster and more accurate for this benchmark.

Asymptotic comparison

For an $N \times N$ grid:
SOR:

$$O(N^2) \text{ per sweep} \times O(N) \text{ sweeps} = O(N^3).$$

FFT/DST direct solver:

$$O(N^2 \log N).$$

Asymptotic speedup:

$$\frac{O(N^3)}{O(N^2 \log N)} = O\left(\frac{N}{\log N}\right).$$

For $N = 4096$:

$$\frac{4096}{\log_2(4096)} \approx 341.$$

Measured speedup is lower because of constants, memory traffic, and implementation overhead.

OpenMP stack issue: why SOR segfaulted

Large local arrays can go on the stack:

```
real(8) :: u(Nx,Ny)
```

For 2026×2026:

$$2026^2 \times 8 \approx 33 \text{ MB}.$$

Linux stack limit is often much smaller:

```
ulimit -s
```

Correct HPC practice:

```
real(8), allocatable :: u(:,:)
allocate(u(Nx,Ny))
```

Large arrays should live on the heap, not the stack.

Summary

Main lessons:

- Jacobi is thread-safe with separate read/write arrays, but slow.
- Red-black SOR is much faster than Jacobi, but still iterative.
- Dirichlet Laplace is diagonalized by sine modes, not periodic modes.
- cuFFT can implement DST-I through odd extension.
- Full 2D odd extension is clear but memory-heavy.
- Batched 1D DSTs give the same mathematics with lower memory and better speed.

Core formula:

$$\widehat{u}_{k,\ell} = \frac{\widehat{f}_{k,\ell}}{\lambda_k^x + \lambda_\ell^y}.$$

One-slide algorithm

Given Dirichlet BCs:

1. Build interior RHS from boundary values.
2. Apply DST-I in x.
3. Apply DST-I in y.
4. Divide by discrete eigenvalues:
$$\lambda_{k,\ell} = \lambda_k^x + \lambda_\ell^y.$$
5. Apply inverse DST-I in y.
6. Apply inverse DST-I in x.
7. Restore boundary values and validate.

This is a direct spectral solve: no convergence loop.